

Sprawozdanie z laboratorium

Techniki programowania I: Języki Maszynowe

Lab2: Pętle i instrukcje warunkowe

18.10.2026

Prowadzący: dr inż. Piotr Witczak

Grupa: 11-INF/B

Kubiak, Łukasz
117841@g.elearn.uz.zgora.pl

1 Wstęp

Celem laboratorium było zapoznanie się z podstawowymi mechanizmami sterowania przebiegiem programu w asemblerze NASM. W ramach zajęć przeanalizowano działanie instrukcji porównania (`cmp`) oraz skoków warunkowych i bezwarunkowych (`jmp`, `je`, `jg`, `jl`, `jge`). Przetestowano również realizację pętli przy użyciu dedykowanej instrukcji `loop` oraz ręcznego sterowania licznikiem. Zwieńczeniem zadań była praktyczna implementacja algorytmu sortowania bąbelkowego dla tablicy danych.

2 Zadania

Zadanie 1: Opisać, jakie zadanie wykonuje program początkowy.

```
global _start

section .text
_start:
   mov rax, 7          ; operand1
   mov rbx, 3          ; operand2

   cmp rax, rbx
   je equal
   jg greater
   jl less

greater:
   mov rdi, 1
   jmp exit_program

equal:
   mov rdi, 0
   jmp exit_program

less:
   mov rdi, -1

exit_program:
   mov rax, 60
   syscall
```

Listing 1: Program początkowy

Program porównuje dwie liczby z rejestrów a i b (dalej nazwanych liczbami a i b). W zależności czy liczba a jest większa, równa lub mniejsza od liczby b, do rejestru d (dalej d) trafia określona wartość.

Dla a > b -> d = 1.

Dla a == b -> d = 0.

Dla a < b -> d = -1.

Jako, że a = 7, a b = 3, d = 1.

Zadanie 2: Korzystając z instrukcji porównania oraz skoków warunkowych, napisać program, który w pętli zwiększa wartość akumulatora o 3 w każdym kroku pętli. Pętla powinna wykonać 10 iteracji.

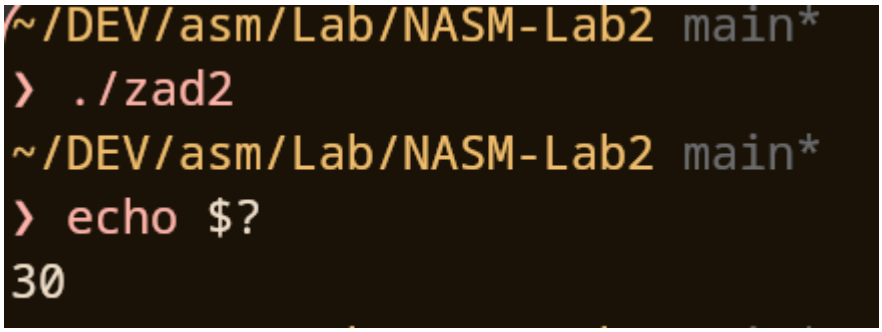
```
global _start

section .text
_start:
  mov rcx, 10 ; licznik, petla ma sie wykonac 10 razy
  xor rax, rax ; zerowanie rax

  _petla_start:
  add rax, 3 ; zwiekszamy wartosc rax o 3
  loop _petla_start
  mov rbx, rax ; przenosimy wartosc rax do rbx
  mov rax, 1 ; sys_exit, koniec programu
  int 0x80 ; wywołanie
```

Listing 2: Kod do zadania 2

Program po wykonaniu zwraca kod 30, czyli wartość akumulatora po wykonaniu 10 iteracji.



Zadanie 3: Zmodyfikować program z zadania 2 w taki sposób, aby w każdym kroku pętli wartość akumulatora była zwiększana o aktualną wartość licznika. Pętla powinna wykonać 10 iteracji, zastosować instrukcję LOOP.

```
global _start

section .text
_start:
  mov rcx, 10 ; licznik, petla ma sie wykonac 10 razy
  xor rax, rax ; zerowanie eax

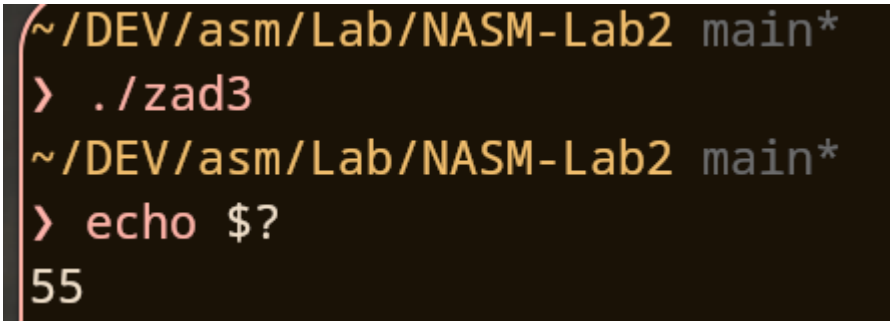
  _petla_start:
  add rax, rcx ; zwiekszamy wartosc licznika
  loop _petla_start
  mov rbx, rax ; przenosimy wartosc eax do ebx
  mov rax, 1 ; sys_exit, koniec programu
  int 0x80 ; wywołanie
```

Listing 3: Zmodyfikowany kod do zadania 3

Licznik z każdą iteracją zmniejsza swoją wartość o 1. Wartością początkową jest 10:

10+9+8+7+6+5+4+3+2+1=55

Program po wykonaniu zwraca kod 55, a więc wynik się zgadza.



Zadanie 4: Napisać program wykonujący algorytm sortowania bąbelkowego dla tablicy zawierającej 5 elementów: `3, 4, 1, 2, 5` (według szablonu).

```
global _start

section .data
   arr db 3, 4, 1, 2, 5

section .text
_start:
   mov al, [arr+1] ; lub movzx rax, byte [arr+1]
   ; kod sortowania

   mov rax, 60
   xor rdi, rdi
   syscall
```

Listing 4: Szablon do zadania 4

```
global _start

section .data
   arr db 3, 4, 1, 2, 5

section .text
_start:
   mov al, [arr+1] ; lub movzx rax, byte [arr+1]

   jmp _again
   _outer_loop:
      mov al, [arr + rsi + 1]
      mov bl, [arr + rsi]

      cmp al, bl
      jge _skip ; jezeli 2 element większy od pierwszego pomijamy zamianę miejscami
      _change:
         mov [arr + rsi], al ; zamiana miejscami
         mov [arr + rsi + 1], bl
         inc rcx ; zwiększ licznik zmian

      _skip:
         inc rsi ; zwiększ liczbę iteracji
         cmp rsi, 4
         jl _outer_loop ; tablica ma 5 elementów więc iteracji nie może być więcej

      _is_sorted:
```

```
        cmp rcx, 0
        jnz _again ; jeżeli wystąpiła conajmniej jedna zamiana miejscami, sprawdź tablicę jeszcze
raz
    mov rax, 60
    xor rdi, rdi
    syscall
    _again:
        xor rsi,rsi ; zeruj licznik iteracji
        xor rcx, rcx ; zeruj licznik zmian
        jmp _outer_loop
```

Listing 5: Sortowanie bąbelkowe

Po pierwszy pełnym wykonaniu pętli `_outer_loop` elementy w tablicy były w następującej kolejności:

```
(gdb) x/5bd &arr
0x402000 <arr>: 3       1       2       4       5
```

Wykonano 2 zamiany miejscami.

Po następnym wykonaniu pętli tablica została posortowana (od najmniejszej do największej wartości) wykonując 2 zmiany:

```
(gdb) x/5bd &arr
0x402000 <arr>: 1       2       3       4       5
```

Za trzecim wykonaniem pętli nie wykonano żadnych zamian miejscami, a więc program się zakończył.

5 Wnioski

W trakcie laboratorium przeanalizowano i zimplementowano podstawowe mechanizmy sterowania przebiegiem programu w asemblerze x86-64.

Na podstawie zrealizowanych zadań przeanalizowano działanie instrukcji porównania (`cmp`) oraz skoków warunkowych (`je`, `jg`, `jl`, `jge`), które umożliwiają rozgałęzianie kodu w zależności od stanu flag procesora. Przetestowano również wykorzystanie instrukcji `loop`, która automatycznie dekrementuje wartość rejestru `rcx` i steruje pętlą do momentu osiągnięcia zera, co ułatwia wykonywanie powtarzalnych operacji arytmetycznych. Zastosowanie indeksowania adresów pamięci (`[arr + rsi]`) w połączeniu z instrukcjami skoków pozwoliło na pomyślną realizację algorytmu sortowania bąbelkowego na tablicy jednowymiarowej.